МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №2

з дисципліни **"Функціональне програмування"**

на тему **"ФУНКЦІЇ ВИЩОГО ПОРЯДКУ, LAMBDA, КОМПОЗИЦІЯ"**

Варіант 13. Доставка посилок

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: опанувати функції вищого порядку, `lambda`, `map` / `filter` / `reduce` та функціональну композицію; побудувати конвеєр перетворень над списком записів без мутації вхідних даних і виконати reduce-агрегацію.

Теоретичні відомості:

Функція вищого порядку (HOF) — це функція, яка приймає інші функції як аргументи або повертає функцію. У Python функції є першокласними значеннями, тому їх можна передавати, повертати і зберігати у структурах. Класичні HOF — `map` (перетворення кожного елемента), `filter` (відбір за предикатом) і `functools.reduce` (згортання послідовності в одне значення); їх генераторними еквівалентами є генераторні вирази, які обчислюються ліниво.

Композиція функцій дозволяє будувати конвеєри без проміжних змінних: `compose(f, g)(x) == f(g(x))` застосовує функції справа наліво і повертає нову функцію, `pipe(x, f, g)` застосовує їх зліва направо й одразу повертає результат, а `juxt(f, g)(x)` повертає кортеж результатів кількох функцій над тим самим значенням. Якщо кожен крок конвеєра має однаковий тип `Iterable[Record] -> Iterable[Record]` і повертає нові записи (`{**r, "поле": значення}`), конвеєр лишається чистим: вхідні дані не змінюються, результат залежить лише від аргументів, а кожен крок можна тестувати окремо.

Хід роботи:

1. У `core.py` реалізував засоби композиції: `compose` (справа наліво, повертає нову функцію), `pipe` (зліва направо через `reduce`) і `juxt` (кортеж результатів

кількох функцій). Типізовано через `TypeVar`, тому конвеєр з однотипних кроків проходить `mypy --strict`:

```python
A = TypeVar("A")
B = TypeVar("B")


# --- композиція ---


def compose(*funcs: Callable[[A], A]) -> Callable[[A], A]:
    """Композиція справа наліво: compose(f, g)(x) == f(g(x))."""

    def _composed(x: A) -> A:
        for f in reversed(funcs):
            x = f(x)
        return x

    return _composed


def pipe(x: A, *funcs: Callable[[A], A]) -> A:
    """Конвеєр зліва направо: pipe(x, f, g) == g(f(x))."""
    return reduce(lambda acc, f: f(acc), funcs, x)


def juxt(*funcs: Callable[[A], B]) -> Callable[[A], tuple[B, ...]]:
    """juxt(f, g)(x) == (f(x), g(x)) — кілька поглядів на одне значення."""
    return lambda x: tuple(f(x) for f in funcs)
```

2. За варіантом 13 описав запис посилки `Parcel` через `TypedDict` (`id`, `sender`, `region`, `delivery_parts`, обчислюване поле `cost`), тип кроку конвеєра `ParcelStep` та структуру результату агрегації `DeliveryStats`:

```python
class Parcel(TypedDict, total=False):
    id: int
    sender: str
    region: str
    delivery_parts: list[float]
    cost: float


ParcelStep = Callable[[Iterable[Parcel]], Iterable[Parcel]]


class DeliveryStats(TypedDict):
    count: float
    delivery_sum: float
    avg_delivery: float
```

3. Реалізував чисті трансформації як фабрики, що повертають функції над `Iterable[Parcel]`: `with_cost` (map: вартість = сума частин), `boost_region` (map: регіональний коефіцієнт лише для заданого регіону), `min_cost` (filter із `lambda`-предикатом), `normalize_senders` (map з `lambda`), `sort_by_cost_desc` (сортування

з двома ключами: вартість спадно, `id` зростом) і `take` (Top-N через `itertools.islice`). Кожен крок повертає нові словники:

```python
def with_cost() -> ParcelStep:
    """map: cost = сума частин вартості доставки (новий запис)."""
    return lambda parcels: (
        {**p, "cost": float(sum(p.get("delivery_parts", []) or []))} for p in parcels
    )


def boost_region(region: str, factor: float) -> ParcelStep:
    """map: регіональний коефіцієнт до вартості лише для заданого регіону."""

    def _boost(parcels: Iterable[Parcel]) -> Iterator[Parcel]:
        for p in parcels:
            cost = float(p.get("cost", 0.0))
            yield {**p, "cost": cost * factor if p.get("region") == region else cost}

    return _boost


def min_cost(threshold: float) -> ParcelStep:
    """filter: лишити посилки з вартістю не нижче порога."""
    return lambda parcels: filter(
        lambda p: float(p.get("cost", 0.0)) >= threshold, parcels
    )


def normalize_senders() -> ParcelStep:
    """map: ім'я відправника у Title Case без зайвих пробілів."""
    return lambda parcels: map(
        lambda p: {**p, "sender": str(p.get("sender", "")).strip().title()}, parcels
    )


def sort_by_cost_desc() -> ParcelStep:
    """sorted з двома ключами: вартість спадно, id зростом."""
    return lambda parcels: sorted(
        parcels, key=lambda p: (-float(p.get("cost", 0.0)), int(p.get("id", 0)))
    )


def take(n: int) -> ParcelStep:
    """Top-N через itertools.islice."""
    return lambda parcels: list(islice(parcels, n))
```

4. Додав reduce-агрегацію `reduce_delivery_stats` з трьома показниками (`count`, `delivery_sum`, `avg_delivery`) — акумулятором є кортеж, щоб не мутувати проміжний стан, — і функцію `build_delivery_pipeline`, яка збирає весь

конвеєр через `compose` і повертає чисту функцію з явними параметрами (Top-N, регіон, коефіцієнт, поріг):



```python
def reduce_delivery_stats(parcels: Iterable[Parcel]) -> DeliveryStats:
    """reduce-агрегація: count, delivery_sum, avg_delivery."""
    count, total = reduce(
        lambda acc, p: (acc[0] + 1, acc[1] + float(p.get("cost", 0.0))),
        parcels,
        (0.0, 0.0),
    )
    return {
        "count": count,
        "delivery_sum": total,
        "avg_delivery": total / count if count else 0.0,
    }


def build_delivery_pipeline(
    top_n: int = 3,
    region: str = "West",
    factor: float = 1.2,
    threshold: float = 0.0,
) -> Callable[[Iterable[Parcel]], list[Parcel]]:
    """Складає конвеєр варіанта 13 через compose; повертає чисту функцію."""
    steps = compose(
        take(top_n),
        sort_by_cost_desc(),
        min_cost(threshold),
        boost_region(region, factor),
        with_cost(),
        normalize_senders(),
    )
    return lambda parcels: list(steps(parcels))
```

5. Базовий приклад з методички (клієнти інтернет-магазину) також реалізував у тому ж стилі: `only_adults`, `with_total`, `boost_city`, `normalize_names`,

`sort_by_total_desc`, `take_records`, `reduce_stats` і `build_pipeline`:

```python
class Record(TypedDict, total=False):
    id: int
    name: str
    age: int
    city: str
    purchases: list[float]
    total: float


RecordStep = Callable[[Iterable[Record]], Iterable[Record]]


class Stats(TypedDict):
    count: float
    sum_total: float
    avg_total: float


def only_adults(min_age: int = 18) -> RecordStep:
    """filter: повнолітні."""
    return lambda recs: (r for r in recs if int(r.get("age", 0)) >= min_age)


def with_total() -> RecordStep:
    """map: total = sum(purchases)."""
    return lambda recs: (
        {**r, "total": float(sum(r.get("purchases", []) or []))} for r in recs
    )


def boost_city(city: str, factor: float) -> RecordStep:
    """map: підсилити total для заданого міста."""

    def _boost(recs: Iterable[Record]) -> Iterator[Record]:
        for r in recs:
            t = float(r.get("total", 0.0))
            yield {**r, "total": t * factor if r.get("city") == city else t}

    return _boost


def normalize_names() -> RecordStep:
    """map: ім'я у Title Case."""
    return lambda recs: (
        {**r, "name": str(r.get("name", "")).strip().title()} for r in recs
    )


def sort_by_total_desc() -> RecordStep:
    """sorted: total спадно, id зростом."""
    return lambda recs: sorted(
        recs, key=lambda r: (-float(r.get("total", 0.0)), r.get("id", 0))
    )


def take_records(n: int) -> RecordStep:
    """Top-N для записів клієнтів."""
    return lambda recs: list(islice(recs, n))
```

```python
def reduce_stats(recs: Iterable[Record]) -> Stats:
    """reduce-arperація: count, sum_total, avg_total."""
    count, total = reduce(
        lambda acc, r: (acc[0] + 1, acc[1] + float(r.get("total", 0.0))),
        recs,
        (0.0, 0.0),
    )
    return {
        "count": count,
        "sum_total": total,
        "avg_total": total / count if count else 0.0,
    }


def build_pipeline(
    top_n: int = 2, city: str = "Delhi", factor: float = 1.1
) -> Callable[[Iterable[Record]], list[Record]]:
    """Конвеєр базового прикладу з методички."""
    steps = compose(
        take_records(top_n),
        sort_by_total_desc(),
        boost_city(city, factor),
        with_total(),
        only_adults(18),
        normalize_names(),
    )
    return lambda recs: list(steps(recs))
```

6. Увесь I/O виніс в `app.py`: читання JSON, розбір аргументів командного рядка через `argparse` (`--data`, `--top`, `--boost-region`, `--factor`, `--min-cost`), функцію `render`, що лише перетворює чистий результат на текст, і друк:

```python
    """IO-адаптер: читає посилки з JSON-файлу."""
    return list(json.loads(path.read_text(encoding="utf-8")))


def render(top: list[Parcel], stats: DeliveryStats) -> str:
    """Перетворює чистий результат на текст (без друку)."""
    lines = [
        f"{p['id']:>3}  {p['sender']:<18} {p['region']:<6} {p['cost']:>8.2f}"
        for p in top
    ]
    return "\n".join(
        [
            *lines,
            f"count={stats['count']:.0f} delivery_sum={stats['delivery_sum']:.2f} "
            f"avg_delivery={stats['avg_delivery']:.2f}",
        ]
    )


def parse_args(argv: list[str]) -> argparse.Namespace:
    """Розбір аргументів CLI."""
    parser = argparse.ArgumentParser(description="Top-N посилок за вартістю доставки")
    parser.add_argument("--data", type=Path, help="JSON зі списком посилок")
    parser.add_argument("--top", type=int, default=3)
    parser.add_argument("--boost-region", default="West")
    parser.add_argument("--factor", type=float, default=1.2)
    parser.add_argument("--min-cost", type=float, default=0.0)
    return parser.parse_args(argv)


def main(argv: list[str]) -> None:
    """Точка входу: читання → чистий конвеєр → друк."""
    args = parse_args(argv)
    parcels = load_parcels(args.data) if args.data else SAMPLE
    pipeline = build_delivery_pipeline(
        args.top, args.boost_region, args.factor, args.min_cost
    )
    top = pipeline(parcels)
    print(render(top, reduce_delivery_stats(top)))


if __name__ == "__main__":
    main(sys.argv[1:])
```

7. Написав pytest-тести у `tests/test_core.py`: порядок Top-N і тай-брейкер за `id` при однаковій вартості, відсутність мутацій вхідного списку, фільтр за порогом, окремі map-кроки, нормалізацію імен і правильність reduce-агрегації (зокрема для порожнього входу):

```python
def test_top_n_order_and_tiebreak(parcels: list[Parcel]) -> None:
    top = build_delivery_pipeline(top_n=3, region="West", factor=1.2)(parcels)
    assert [p["id"] for p in top] == [2, 1, 4]
    assert top[1]["cost"] == pytest.approx(84.0)
    assert top[2]["cost"] == pytest.approx(84.0)


def test_tiebreak_by_id_when_costs_equal(parcels: list[Parcel]) -> None:
    top = build_delivery_pipeline(top_n=5, region="West", factor=1.2)(parcels)
    costs = [(p["cost"], p["id"]) for p in top]
    assert costs == sorted(costs, key=lambda c: (-c[0], c[1]))
    assert [p["id"] for p in top if p["cost"] == 84.0] == [1, 4, 5]


def test_no_mutation(parcels: list[Parcel]) -> None:
    original = deepcopy(parcels)
    build_delivery_pipeline(top_n=5, threshold=50.0)(parcels)
    reduce_delivery_stats(with_cost()(parcels))
    assert parcels == original


def test_threshold_filter(parcels: list[Parcel]) -> None:
    top = build_delivery_pipeline(top_n=10, region="None", factor=1.0, threshold=84.0)(
        parcels
    )
    assert [p["id"] for p in top] == [2, 4, 5]


def test_with_cost_and_boost(parcels: list[Parcel]) -> None:
    costed = list(with_cost()(parcels))
    assert [p["cost"] for p in costed] == [70.0, 120.0, 15.5, 84.0, 84.0]
    boosted = list(boost_region("West", 2.0)(costed))
    assert [p["cost"] for p in boosted] == [140.0, 120.0, 31.0, 84.0, 84.0]
    assert "cost" not in parcels[0]


def test_normalize_senders(parcels: list[Parcel]) -> None:
    names = [p["sender"] for p in normalize_senders()(parcels)]
    assert names[:2] == ["Ivan Petrenko", "Olha Shevchenko"]


def test_reduce_delivery_stats(parcels: list[Parcel]) -> None:
    stats = reduce_delivery_stats(with_cost()(parcels))
    assert stats["count"] == 5
    assert stats["delivery_sum"] == pytest.approx(373.5)
    assert stats["avg_delivery"] == pytest.approx(74.7)
    assert reduce_delivery_stats([]) == {
        "count": 0.0,
        "delivery_sum": 0.0,
        "avg_delivery": 0.0,
    }

```

8. Додав тести композиції: результат `compose(...)` збігається з ручним вкладеним ланцюжком викликів, `pipe` і `juxt` працюють як очікується, а базовий

конвеєр з методички дає правильний Top-N, статистику й не мутує вхід:

```python
def test_composition_equals_manual_steps(parcels: list[Parcel]) -> None:
    manual = take(2)(
        sort_by_cost_desc()(
            min_cost(80.0)(boost_region("East", 1.5)(with_cost()(parcels)))
        )
    )
    composed = compose(
        take(2),
        sort_by_cost_desc(),
        min_cost(80.0),
        boost_region("East", 1.5),
        with_cost(),
    )
    assert list(composed(parcels)) == list(manual)
    assert [p["id"] for p in manual] == [2, 5]


def test_pipe_and_juxt() -> None:
    assert pipe(3, lambda x: x + 1, lambda x: x * 2) == 8
    assert compose(lambda x: x * 2, lambda x: x + 1)(3) == 8

    def length(xs: list[int]) -> int:
        return len(xs)

    def total(xs: list[int]) -> int:
        return sum(xs)

    assert juxt(length, total)([3, 1, 2]) == (3, 6)


# --- базовий приклад з методички ---


def test_base_pipeline_top_n(records: list[Record]) -> None:
    top = build_pipeline(top_n=2, city="Delhi", factor=1.1)(records)
    assert [r["id"] for r in top] == [1, 3]
    assert top[0]["name"] == "Amit"
    assert top[0]["total"] == pytest.approx(165.0)


def test_base_reduce_stats(records: list[Record]) -> None:
    stats = reduce_stats(build_pipeline(top_n=10)(records))
    assert stats["count"] == 3
    assert stats["sum_total"] == pytest.approx(165.0 + 160.0 + 110.0)
    assert stats["avg_total"] == pytest.approx(435.0 / 3)


def test_base_no_mutation(records: list[Record]) -> None:
    original = deepcopy(records)
    build_pipeline()(records)
    assert records == original
```

9. У `tests/test_properties.py` додав property-based тести на Hypothesis: для випадкових списків посилок Top-N завжди є префіксом повного відсортованого

результату, конвеєр не змінює вхід, а статистики узгоджені між собою:

```python
parcels_st = st.lists(
    st.fixed_dictionaries(
        {
            "id": st.integers(min_value=1, max_value=10_000),
            "sender": st.text(min_size=1, max_size=12),
            "region": st.sampled_from(["West", "East", "North"]),
            "delivery_parts": st.lists(
                st.floats(min_value=0, max_value=1e4, allow_nan=False), max_size=5
            ),
        }
    ),
    max_size=30,
)


@given(parcels_st, st.integers(min_value=0, max_value=10))
def test_top_n_is_prefix_of_sorted(items: list[Parcel], n: int) -> None:
    full = build_delivery_pipeline(top_n=len(items) + 1)(items)
    top = build_delivery_pipeline(top_n=n)(items)
    assert top == full[:n]
    keys = [(-p["cost"], p["id"]) for p in full]
    assert keys == sorted(keys)


@given(parcels_st)
def test_pipeline_does_not_mutate(items: list[Parcel]) -> None:
    original = deepcopy(items)
    build_delivery_pipeline(top_n=5, threshold=10.0)(items)
    assert items == original


@given(parcels_st)
def test_stats_are_consistent(items: list[Parcel]) -> None:
    costed = list(with_cost()(items))
    stats = reduce_delivery_stats(costed)
    assert stats["count"] == len(items)
    assert abs(stats["delivery_sum"] - sum(p["cost"] for p in costed)) < 1e-6
    if items:
        assert (
            abs(stats["avg_delivery"] * stats["count"] - stats["delivery_sum"]) < 1e-6
        )
```

10. Запустив CLI: з п'яти посилок із коефіцієнтом 1.2 для регіону West і порогом 50 відібрано три найдорожчі; при однаковій вартості 84.00 порядок визначає `id`. Другий запуск читає дані з `sample_data.json`:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python app.py --top 3 --boost-region West --fact
2  Olha Shevchenko    East     120.00
  1  Ivan Petrenko      West      84.00
  4  Maria Koval        North     84.00
count=3 delivery_sum=288.00 avg_delivery=96.00

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python app.py --data sample_data.json --top 2
2  Olha Shevchenko    East     120.00
  1  Ivan Petrenko      West      84.00
count=2 delivery_sum=204.00 avg_delivery=102.00
```

11. Перевірив проєкт автоматичними інструментами: усі 15 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 15 items

tests\test_core.py::test_top_n_order_and_tiebreak PASSED                 [  6%]
tests\test_core.py::test_tiebreak_by_id_when_costs_equal PASSED          [ 13%]
tests\test_core.py::test_no_mutation PASSED                              [ 20%]
tests\test_core.py::test_threshold_filter PASSED                         [ 26%]
tests\test_core.py::test_with_cost_and_boost PASSED                      [ 33%]
tests\test_core.py::test_normalize_senders PASSED                        [ 40%]
tests\test_core.py::test_reduce_delivery_stats PASSED                    [ 46%]
tests\test_core.py::test_composition_equals_manual_steps PASSED          [ 53%]
tests\test_core.py::test_pipe_and_juxt PASSED                            [ 60%]
tests\test_core.py::test_base_pipeline_top_n PASSED                      [ 66%]
tests\test_core.py::test_base_reduce_stats PASSED                        [ 73%]
tests\test_core.py::test_base_no_mutation PASSED                         [ 80%]
tests\test_properties.py::test_top_n_is_prefix_of_sorted PASSED          [ 86%]
tests\test_properties.py::test_pipeline_does_not_mutate PASSED           [ 93%]
tests\test_properties.py::test_stats_are_consistent PASSED               [100%]

============================== 15 passed in 0.84s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python -m mypy .
Success: no issues found in 5 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab02> python -m black --check .
All done! ✨ 🍰 ✨
5 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я побудував функціональний конвеєр перетворень над записами посилок (варіант 13) з `map`, `filter` і генераторних виразів, зібрав його через власну композицію `compose` з однотипних чистих кроків і виконав reduce-агрегацію з трьома показниками. `lambda` використано у предикаті фільтра, map-кроках і ключі сортування з двома полями; жоден крок не змінює вхідні записи, а I/O ізольовано в `app.py`. Юніт- і property-тести, а також mypy, ruff і black підтвердили коректність Top-N, відсутність мутацій і правильність агрегації та композиції.